

南开大学
Nankai University

南　　开　　大　　学

计　算　机　学　院

自然语言处理实验报告

作业一：文本分类实现

姓名：彭坤

年级：2023 级

专业：计算机科学与技术

指导教师：曾庆锴

2026 年 10 月 10 日

目录

一、 实验目的与任务概述 **2**
（一） 实验目的 . . . 2
（二） 任务概述 . . . 2

二、 数据集与统一实验设置 **2**
（一） 数据集与类别分布 . . . 2
（二） 统一数据划分与复现机制 . . . 3
（三） 预处理与训练约束 . . . 3
（四） 评价指标 . . . 3
（五） 运行环境与基本参数 . . . 4

三、 Task 1：基于词袋的文本分类 **5**
（一） 词表构建与特征提取 . . . 5
（二） 两种词袋表示 . . . 5
1. Binary Bag of Words . . . 5
2. Word Frequency . . . 5
（三） Logistic Regression 与实现流程 . . . 5
（四） 实验结果 . . . 6
（五） 类别表现与结果分析 . . . 7

四、 Task 2：基于平均词向量的文本分类 **7**
（一） 平均词向量文档表示 . . . 7
（二） 三组词向量的来源与构建 . . . 8
1. 预训练 GloVe . . . 8
2. 在 AG News 上训练 Word2Vec . . . 8
3. 在 NYT 训练集上训练 Word2Vec . . . 9
（三） Word2Vec 训练设置与分类流程 . . . 9
（四） 实验结果与词覆盖率 . . . 9
（五） 类别表现 . . . 10
（六） 结果分析 . . . 11

五、 Task 3：基于 BERT 的文本分类 **11**
（一） 模型结构与分类目标 . . . 11
（二） 分词、截断与输入构建 . . . 12
（三） 微调设置与训练流程 . . . 12
（四） 训练过程与验证表现 . . . 13
（五） 最终测试结果与类别分析 . . . 13
（六） 与传统方法的比较及局限 . . . 14

一、 实验目的与任务概述

(一) 实验目的

文本分类的目标是根据文档内容预测其所属类别，是自然语言处理中的基本任务。文本表示方法决定了分类模型能够利用的信息：词袋表示直接记录词语的出现情况，词向量将单词映射到稠密的语义空间，而预训练语言模型能够结合上下文生成文本表示。本实验以新闻类别预测为具体任务，在统一的数据划分和评价标准下，考察不同文本表示方法的分类效果。

本实验主要包含三个方面的学习目标。首先，理解二值词袋、词频、静态词向量与 BERT 表示的基本思想，掌握从原始新闻文本到分类模型输入的转换过程。其次，完成数据读取、训练与评价的实验流程，实践 Logistic Regression 分类器、Word2Vec 词向量训练以及 BERT 微调，并通过训练集与评价集的分离控制数据泄漏。最后，使用 Accuracy 和 Macro-F1 衡量模型性能，结合类别分布、每类指标及混淆矩阵分析不同方法的表现差异。

(二) 任务概述

本实验以 New York Times（NYT）新闻数据集为主要分类数据，完成三个任务，共六组实验：

1. **Task 1：基于词袋的文本分类**。分别采用 Binary Bag of Words 和 Word Frequency 构建文档表示，前者记录单词是否出现，后者记录单词出现的次数。两种表示均使用 Logistic Regression 进行分类，以比较词频信息对分类效果的影响。

2. **Task 2：基于平均词向量的文本分类**。分别使用预训练 GloVe、在 AG News 文本上训练的 Word2Vec，以及在 NYT 训练集文本上训练的 Word2Vec。三组实验统一采用 100 维词向量，通过有效词向量的算术平均得到文档向量，并使用 Logistic Regression 进行分类，以考察不同词向量来源的影响。

3. **Task 3：基于 BERT 的文本分类**。使用 `google-bert/bert-base-uncased` 进行微调，最大输入序列长度固定为 64，训练轮数固定为 3。BERT 编码器与分类头共同参与训练，在完成第三轮训练后使用最终模型评价测试集性能。

六组实验均在同一份 NYT 测试集上报告 Accuracy 和 Macro-F1。后续分析围绕文本表示方式、词向量训练语料和模型训练设置展开，以实际实验结果讨论各方法在本数据集上的表现。

二、 数据集与统一实验设置

(一) 数据集与类别分布

本实验使用作业提供的 NYT 与 AG News 两份 CSV 数据。NYT 包含 `text` 和 `label` 两列，分别表示新闻正文与新闻类别，共有 11,519 条记录，类别为商业（`business`）、政治（`politics`）和体育（`sports`）。AG News 包含 `text` 列，共有 90,000 条文本，仅用于 Task 2 中 AG Word2Vec 的词向量训练，其文本不进入 NYT 分类任务的验证集或测试集。

数据读取遵循 CSV 中原始记录的顺序，保留新闻文本，不静默删除或重排样本；遇到空文本或空标签时明确报错。NYT 的类别数量及各子集分布如表 1 所示。

体育类样本约占全部数据的 75%，明显多于商业和政治两类。因此，整体准确率容易受到多数类别表现的影响，需要同时使用对各类别赋予相同权重的 Macro-F1，并结合每类指标分析模型的分类能力。

表 1: NYT 数据集的类别分布与划分结果

类别	全部数据	训练集	验证集	测试集
商业（business）	1,429	1,143	143	143
政治（politics）	1,451	1,161	145	145
体育（sports）	8,639	6,911	864	864
合计	11,519	9,215	1,152	1,152

(二) 统一数据划分与复现机制

NYT 按训练集 80%、验证集 10%、测试集 10% 的比例进行随机划分，随机种子统一设置为 42。实现时采用两次分层随机抽样：首先保留约 80% 的样本作为训练集，再将剩余约 20% 的样本均分为验证集和测试集。两次抽样均依据类别标签进行分层，使各子集的类别比例接近完整数据集。由于样本数取整，三个子集的实际数量分别为 9,215、1,152 和 1,152。

三个任务通过同一份 `data/split_indices.json` 共享划分。该文件保存各子集的原始记录索引、样本总数、随机种子和原始 NYT 文件的 SHA-256 指纹。索引从 0 开始，表示不含表头的数据记录序号。各任务读取相同顺序的 NYT 数据，再依据这些索引选取样本，因此不同文本表示方法对应的是完全相同的训练、验证和测试新闻。

共享的数据处理逻辑位于 `src/common.py`。任意任务在划分文件缺失时均可按统一规则生成并保存划分，已有文件则校验后复用，因而三个任务可以独立启动。读取时检查数据指纹、样本数量、索引起点，以及索引是否存在重复、遗漏或越界，避免因数据替换、重排或划分损坏而破坏实验的可比性。

(三) 预处理与训练约束

Task 1 与 Task 2 使用相同的词元提取规则：将文本转为小写，以连续的字母、数字及下划线字符构成词元，保留单字符词元，不移除停用词，也不进行词干化。Task 3 使用 BERT 自带的 uncased WordPiece tokenizer，按照模型要求处理特殊词元、截断、填充和 attention mask。三类方法在文本编码方式上各有区别，但使用的原始新闻及所属子集保持一致。

为控制数据泄漏，Task 1 的词表仅从 NYT 训练集建立，所有 Logistic Regression 分类器仅使用训练集特征与标签进行拟合。Task 2 的 NYT Word2Vec 仅使用 NYT 训练集文本训练；AG Word2Vec 使用提供的 AG News 文本训练；GloVe 使用已有的外部预训练词向量。NYT 验证集和测试集均不参与 Word2Vec 参数训练。Task 3 的 BERT 参数更新同样只使用训练集，验证集与测试集不参与反向传播。

本实验采用预先固定的参数配置。验证集用于记录模型表现，BERT 在每轮训练后进行验证；最终结果统一在测试集上评价，未依据测试集指标调整超参数或选择模型。BERT 使用完成 3 轮训练后的最终模型进行测试，不从不同轮次中依据测试结果挑选模型。

(四) 评价指标

设测试集包含 N 条样本，第 i 条样本的真实类别为 y_i，预测类别为 $\hat{y}_i$，则准确率定义为

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{y}_i = y_i), \tag{1}$$

其中，$\mathbb{I}(\cdot)$ 为指示函数，条件成立时取 1，否则取 0。Accuracy 表示预测正确的样本占全部测试样本的比例。

对类别 k，将该类视为正类、其余类别视为负类，分别记真正例、假正例和假负例数量为 TP_k、FP_k 和 FN_k。该类的精确率、召回率和 F1 定义为

$$P_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FP}_k}, \tag{2}$$

$$R_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FN}_k}, \tag{3}$$

$$F_{1,k} = \frac{2P_kR_k}{P_k + R_k}. \tag{4}$$

F1 是精确率和召回率的调和平均。对 K 个类别分别计算 F1 后，宏平均 F1 为

$$\text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K} F_{1,k}, \qquad K = 3. \tag{5}$$

计算中若相关分母为 0, 则将对应指标记为 0, 与代码中的 `zero_division=0` 设置一致。Macro-F1 对三个类别赋予相同权重，有助于观察模型在商业、政治等较少样本类别上的表现。

除两个主要指标外，实验还保存每类 Precision、Recall、F1、Support 和混淆矩阵。混淆矩阵的行表示真实类别，列表示预测类别；行列顺序由结果文件中的 `class_order` 指定，本实验为商业、政治、体育。对角线记录预测正确的样本数量，非对角线记录不同类别之间的误分类数量，为后续误差分析提供依据。

（五） 运行环境与基本参数

Task 1、Task 2 的传统模型在 CPU 上运行, Task 3 的正式实验使用 NVIDIA Tesla T4 GPU, 并采用 FP16 混合精度。表 2 列出完整实验记录中的主要软件版本；传统模型与 BERT 分别在各自环境中运行。

表 2: 正式实验的主要运行环境

项目	Task 1 / Task 2	Task 3
计算设备	CPU	NVIDIA Tesla T4
Python	3.12.14	3.13.15
NumPy	2.5.3	2.1.3
scikit-learn	1.8.0	1.6.1
Gensim	4.4.0（Task 2）	—
PyTorch	—	2.11.0+cu130
Transformers	—	4.57.1
BERT 计算精度	—	FP16 混合精度

各任务的训练随机种子均为 42，数值计算线程数设为 4。Task 1 与 Task 2 的分类器统一使用 Logistic Regression，采用 `lbfgs` 求解器、默认 L2 正则化，$C = 1.0$，最大迭代次数为 2,000；不设置类别权重。Task 2 的三组词向量维度均为 100。

BERT 的最大序列长度为 64，训练轮数为 3，训练 batch size 为 16，评价 batch size 为 32，学习率为 2×10^{-5}，采用 AdamW 优化器，权重衰减系数为 0.01，并在总更新步数的前 10% 进行学习率预热。各方法更具体的表示构建与训练过程将在相应实验章节中展开。

三、 Task 1：基于词袋的文本分类

Task 1 分别使用 Binary Bag of Words 与 Word Frequency 表示新闻文本，并使用相同配置的 Logistic Regression 分类器进行训练。两组实验采用第 二 节中的共享数据划分，文本预处理、词表构建规则和分类器参数保持一致，仅改变单词特征的取值方式，以考察是否保留出现次数对分类结果的影响。

(一) 词表构建与特征提取

实验使用 scikit-learn 的 `CountVectorizer` 提取单词特征。文本转为小写后，按照统一分词规则提取单个词元，不构建二元或更高阶的词组特征，也不限制词表大小。词表仅从 9,215 条训练文本中建立，最终包含 61,021 个不同词元。两种表示使用相同的训练文本和词元规则，因此词表及特征维度一致。

设训练词表为 $V = (v_1, v_2, \ldots, v_{|V|})$，每篇文档均映射为 $|V|$ 维向量。本实验训练集特征矩阵的形状为 $9,215 \times 61,021$，验证集和测试集的特征矩阵形状均为 $1,152 \times 61,021$。每篇新闻通常只包含词表中的一部分单词，因而特征矩阵采用稀疏存储，避免存储大量为零的元素。

验证集和测试集通过训练阶段得到的词表进行转换，其中未出现在训练词表中的词元会被忽略，不为其扩展新的特征维度。所有特征均直接输入分类器，不使用 TF-IDF，也不对词频进行额外归一化。

(二) 两种词袋表示

1. Binary Bag of Words

二值词袋仅记录词元是否在文档中出现。设 $c_j(d)$ 表示词元 v_j 在文档 d 中的出现次数，则第 j 维特征为

$$x_j^{\text{bin}}(d) = \begin{cases} 1, & c_j(d) > 0, \\ 0, & c_j(d) = 0. \end{cases} \tag{6}$$

该方法通过 `CountVectorizer(binary=True)` 实现，同一单词出现一次或多次均对应特征值 1。它保留单词的出现信息，适合考察单词是否出现本身能够提供多少类别判别信息。

2. Word Frequency

词频表示直接记录每个词元在文档中的原始出现次数，即

$$\mathbf{x}^{\text{freq}}(d) = \big(c_1(d), c_2(d), \ldots, c_{|V|}(d)\big). \tag{7}$$

该方法通过 `CountVectorizer(binary=False)` 实现。以词表 $V = (\texttt{apple}, \texttt{banana})$ 和文本 `apple apple banana` 为例，二值词袋向量为 $(1, 1)$，词频向量为 $(2, 1)$。词频表示保留了重复出现的信息，但其数值也会随词语重复程度及文档长度发生变化。

两种表示均不保留词元的先后顺序，属于基于词项统计的文档表示。因此，这一对照实验比较的是词语出现次数的作用，而不是不同语序或上下文建模能力的差异。

(三) Logistic Regression 与实现流程

NYT 包含三个类别，本实验使用多分类 Logistic Regression。对于文档特征 $\mathbf{x}$，类别 k 的预测概率可写为

$$p(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^{\mathsf{T}}\mathbf{x} + b_k)}{\sum_{r=1}^{3} \exp(\mathbf{w}_r^{\mathsf{T}}\mathbf{x} + b_r)}, \tag{8}$$

模型选择概率最大的类别作为预测结果。分类器使用 `lbfgs` 求解器与默认 L2 正则化，$C = 1.0$，最大迭代次数为 2,000，随机种子为 42，不设置类别权重。其中，C 是正则化强度的倒数，值越小，对模型权重的约束越强。

源码通过 `Pipeline` 将文本向量化器与分类器组合起来。对训练集调用 `fit` 时，先建立词表并生成稀疏特征矩阵，再拟合分类器；对验证集和测试集调用 `predict` 时，使用已拟合的词表与分类器完成转换和预测。代码 1 展示了两组实验共用的核心训练流程，其中 `binary` 分别取 `True` 和 `False`。

```
pipeline = Pipeline([
    ("vectorizer", CountVectorizer(
        binary=binary,
        lowercase=True,
        token_pattern=TOKEN_PATTERN,
        dtype=np.float64,
    )),
    ("classifier", LogisticRegression(
        C=1.0,
        solver="lbfgs",
        max_iter=2000,
        random_state=42,
    )),
])
pipeline.fit(texts[split["train"]], labels[split["train"]])
```

Listing 1: Task 1 的词袋表示与分类器训练流程

训练完成后，程序分别对验证集和测试集计算 Accuracy 与 Macro-F1，同时保存每类分类指标、混淆矩阵和逐条测试预测。模型以包含向量化器和分类器的完整 Pipeline 形式保存，以便加载后使用同一词表处理新文本。训练期间记录收敛警告及实际迭代次数，两组正式实验均已收敛。

（四） 实验结果

表 3 给出两种表示在验证集与测试集上的结果。指标保留四位小数。

表 3: Task 1 的验证集与测试集分类结果

方法	验证集 Accuracy	验证集 Macro-F1	测试集 Accuracy	测试集 Macro-F1
Binary BoW	0.9809	0.9551	0.9887	0.9718
Word Frequency	0.9835	0.9626	0.9887	0.9729

两种方法在测试集上均正确分类 1,139 条新闻，错误分类 13 条，准确率均为 $1,139/1,152 \approx 0.9887$。Word Frequency 的测试集 Macro-F1 比 Binary BoW 高约 0.0011，验证集上的两个指标也略高。两种方法的正确预测总数相同，但不同类别的精确率与召回率存在差异，因此宏平均 F1 并不相同。

(五) 类别表现与结果分析

表 4 列出两种方法的每类指标。体育类的 F1 均超过 0.998, 商业和政治两类的 F1 约为 0.95–0.96。较少样本类别的分类效果低于体育类，使得两组实验的 Macro-F1 均低于整体 Accuracy, 也说明仅观察准确率不足以完整描述三个类别的表现。

表 4: Task 1 的测试集每类分类指标

方法	类别	Precision	Recall	F1	样本数
Binary BoW	商业	0.9577	0.9510	0.9544	143
	政治	0.9589	0.9655	0.9622	145
	体育	0.9988	0.9988	0.9988	864
Word Frequency	商业	0.9333	0.9790	0.9556	143
	政治	0.9856	0.9448	0.9648	145
	体育	0.9988	0.9977	0.9983	864

按照商业、政治、体育的顺序排列行列，两种方法的混淆矩阵分别为

$$\mathbf{M}_{\text{bin}} = \begin{bmatrix} 136 & 6 & 1 \\ 5 & 140 & 0 \\ 1 & 0 & 863 \end{bmatrix}, \qquad \mathbf{M}_{\text{freq}} = \begin{bmatrix} 140 & 2 & 1 \\ 8 & 137 & 0 \\ 2 & 0 & 862 \end{bmatrix}. \tag{9}$$

其中，行表示真实类别，列表示预测类别。Binary BoW 将 6 条商业新闻预测为政治、5 条政治新闻预测为商业；Word Frequency 对应的数量分别为 2 条和 8 条。两种方法的错误均主要集中在商业与政治之间，涉及体育类的误分类较少。

保留词频后，正确识别的商业新闻由 136 条增加到 140 条，其召回率提高，但被预测为商业的其他类别样本也有所增加，因此商业类的精确率下降。政治类呈现相反的变化：预测为政治的错误样本减少，精确率提高，但正确识别的政治新闻由 140 条减少到 137 条，召回率下降。商业与政治两类的 F1 均略有提高，体育类的 F1 略有下降，最终得到略高的 Macro-F1。这表明词频特征改变了不同类别之间的精确率与召回率权衡，而非使各项指标同时提高。

在当前 NYT 数据及固定划分下, 两种词袋表示配合线性分类器均取得了较高的测试准确率, 说明训练词表中的词项特征具有较强的类别区分能力。

四、 Task 2：基于平均词向量的文本分类

Task 2 使用静态词向量表示单词，再通过文档内有效词向量的平均得到新闻表示。实验分别采用预训练 GloVe、AG News Word2Vec 和 NYT Word2Vec，三组词向量均为 100 维，分类器统一使用 Logistic Regression。三组实验使用相同的 NYT 数据划分、分词方式与文档向量构建规则，主要比较词向量来源对分类效果的影响。

(一) 平均词向量文档表示

设文档 d 分词后得到词元序列 $(w_1, w_2, \ldots, w_m)$, 词向量词表为 V_e, 单词 w 的向量为 $\mathbf{e}(w) \in \mathbb{R}^{100}$。定义有效词元位置集合为

$$\mathcal{I}(d) = \{i \mid w_i \in V_e,\ 1 \le i \le m\}. \tag{10}$$

文档向量由有效词向量的算术平均得到：

$$\mathbf{z}(d) = \begin{cases} \dfrac{1}{|\mathcal{I}(d)|} \displaystyle\sum_{i \in \mathcal{I}(d)} \mathbf{e}(w_i), & |\mathcal{I}(d)| > 0, \\ \mathbf{0}, & |\mathcal{I}(d)| = 0. \end{cases} \tag{11}$$

平均过程按词元的出现位置计算，重复词重复参与平均。例如，当 `apple` 和 `banana` 均有对应向量时，文本 `apple apple banana` 的文档向量为 $(2\mathbf{e}(\texttt{apple}) + \mathbf{e}(\texttt{banana}))/3$。未知词不参与求和，也不计入分母；没有有效词的文档使用 100 维零向量，并记录其数量。

三组实验的训练集特征矩阵形状均为 $9,215 \times 100$，验证集和测试集均为 $1,152 \times 100$。平均向量以双精度浮点数计算并直接输入分类器，不进行额外标准化、L2 归一化或 TF-IDF 加权。代码 2 展示平均文档向量的核心实现。

```
features = np.zeros((len(documents), 100), dtype=np.float64)
for row, tokens in enumerate(documents):
    indices = [
        vectors.key_to_index[token]
        for token in tokens
        if token in vectors.key_to_index
    ]
    if indices:
        features[row] = vectors.vectors[indices].mean(
            axis=0, dtype=np.float64
        )
```

Listing 2: 有效词向量的平均与未知词处理

(二) 三组词向量的来源与构建

1. 预训练 GloVe

GloVe 通过语料中的全局词共现统计学习单词表示。本实验使用公开的 `glove.6B.100d.txt`，读取已有的 100 维向量，不在 NYT 上重新训练或微调 GloVe。为减少存储和内存占用，程序逐行读取文件，仅保留 NYT 文档可能查询到的词向量，最终载入 59,707 个词。

这里的 59,707 是本实验实际保留的向量数量，并非原始 GloVe 完整词表的大小。筛选操作不改变预训练向量的数值，验证集或测试集中出现的词只要存在于外部预训练词表中，也可以直接查询对应向量。随后依据式 11 构建文档表示。

2. 在 AG News 上训练 Word2Vec

该组实验使用提供的全部 90,000 条 AG News 文本自行训练 Word2Vec，不使用 NYT 标签参与词向量学习。完成统一分词后，该语料包含 3,581,128 个词元出现位置；经过最低词频阈值筛选，最终得到 37,062 个单词的词向量。

AG News 通过可重复遍历的 CSV 语料迭代器提供给训练程序，每次读取一条新闻并分词，使词表构建和多轮训练能够重复遍历语料，无需同时保存全部文本的分词结果。训练得到的向量用于表示 NYT 新闻，Logistic Regression 的拟合与最终评价仍分别使用 NYT 训练集和测试集。

3. 在 NYT 训练集上训练 Word2Vec

该组实验只使用 9,215 条 NYT 训练新闻学习词向量，验证集和测试集均不参与 Word2Vec 的词表构建或参数更新。训练语料分词后包含 6,126,120 个词元出现位置，最低词频阈值筛选后的词表大小为 43,434。

与 AG News 相比，NYT 训练语料的文档数更少，但分词后的词元总数更多，因此文档数量不能单独反映两份训练语料的规模。两组 Word2Vec 的训练参数保持一致，语料来源、文本长度分布及词项分布随数据集发生变化。

(三) Word2Vec 训练设置与分类流程

两组 Word2Vec 均采用 Gensim 实现的 Skip-gram 模型，以中心词预测邻近的上下文词，并使用负采样进行训练。对于中心词 w 与真实上下文词 c，负采样目标的一项可以写为

$$\log \sigma(\mathbf{u}_c^{\mathsf{T}} \mathbf{v}_w) + \sum_{\ell=1}^{q} \mathbb{E}_{n_\ell \sim P_n} \left[\log \sigma(-\mathbf{u}_{n_\ell}^{\mathsf{T}} \mathbf{v}_w) \right], \tag{12}$$

其中，$\mathbf{v}_w$ 和 $\mathbf{u}_c$ 分别为中心词与上下文词的向量，σ 为 sigmoid 函数，P_n 为负样本分布，$q = 5$ 为负样本数。训练提高真实中心词与上下文组合的得分，同时降低与负样本组合的得分。

表 5 列出两组 Word2Vec 共用的主要参数。高频词下采样用于词向量训练；构建 NYT 文档向量时，仍按所有有效词元的出现次数进行平均，不再次执行该下采样过程。训练采用单线程，并使用固定的词初始化哈希函数，以减少并行更新顺序和 Python 进程随机哈希对复现的影响。

表 5: 两组 Word2Vec 的主要训练参数

参数	设置
模型结构	Skip-gram，`sg=1`
词向量维度	`vector_size=100`
最大上下文窗口	`window=5`
最低词频	`min_count=2`
负样本数	`negative=5`
高频词下采样阈值	`sample=1e-3`
训练轮数	`epochs=5`
随机种子	`seed=42`
训练线程数	`workers=1`
初始化哈希	使用单词 UTF-8 编码的 CRC32 值

三组实验首先加载或训练词向量，再分别生成 NYT 训练、验证和测试集的文档向量。分类器配置与第 三 节一致，使用 `lbfgs`、默认 L2 正则化、$C = 1.0$、最大迭代次数 2,000 和随机种子 42，不设置类别权重。分类器仅使用 NYT 训练集特征及标签进行拟合，词向量不会在这一阶段根据分类损失继续更新。

训练完成后，程序保存词向量与分类器，以及验证集和测试集指标、逐条预测、每类分类报告、混淆矩阵和词覆盖情况。三组正式实验的分类器均已收敛。

(四) 实验结果与词覆盖率

表 6 给出三组实验的分类结果，精确数值来自 `results/task2/results.json`。GloVe 在验证集与测试集的两个指标上均最高；NYT Word2Vec 的结果高于 AG Word2Vec。

表 6: Task 2 的验证集与测试集分类结果

方法	验证集 Accuracy	验证集 Macro-F1	测试集 Accuracy	测试集 Macro-F1
GloVe	0.9774	0.9500	0.9852	0.9646
AG Word2Vec	0.9653	0.9228	0.9757	0.9410
NYT Word2Vec	0.9679	0.9304	0.9818	0.9542

GloVe、AG Word2Vec 和 NYT Word2Vec 分别正确分类 1,135、1,124 和 1,131 条测试新闻，对应错误数量为 17、28 和 21。NYT Word2Vec 相比 AG Word2Vec 的测试 Accuracy 提高约 0.0061，Macro-F1 提高约 0.0132；GloVe 相比 NYT Word2Vec 的两个指标分别高约 0.0035 和 0.0104。

为观察词表对 NYT 文本的覆盖程度，程序按词元出现次数统计覆盖率。对某个子集 D，覆盖率定义为

$$\mathrm{Coverage}(D) = \frac{\sum_{d \in D} |\mathcal{I}(d)|}{\sum_{d \in D} |d|}, \tag{13}$$

其中，$|d|$ 是文档分词后的词元数量，重复词重复计数。这一指标表示能够找到词向量的词元出现次数比例，并非不同单词种类的覆盖比例。

表 7: Task 2 的实际向量词表与测试集词覆盖情况

方法	实际向量词表大小	测试集有效词元数	覆盖率	无有效词文档数
GloVe	59,707	780,519	99.79%	0
AG Word2Vec	37,062	761,274	97.33%	0
NYT Word2Vec	43,434	776,592	99.29%	0

测试集共包含 782,173 个词元出现位置。三组方法均不存在无有效词的测试文档，其训练集和验证集也均没有此类文档，因此本次结果差异并非由整篇新闻被表示为零向量引起。GloVe 的覆盖率最高，NYT Word2Vec 次之，AG Word2Vec 最低；其排序与整体分类结果一致。不过，覆盖率只描述可查询向量的数量比例，不能直接衡量向量的语义质量，也不能单独解释性能差异。

（五） 类别表现

表 8 给出各类别的测试 F1。三组词向量在体育类上的 F1 均超过 0.996，差异主要体现在商业与政治两类。GloVe 在这两类上的 F1 最高，而 NYT Word2Vec 在体育类上的 F1 略高于 GloVe。

表 8: Task 2 的测试集每类 F1

类别	GloVe	AG Word2Vec	NYT Word2Vec	样本数
商业	0.9444	0.9072	0.9296	143
政治	0.9517	0.9193	0.9347	145
体育	0.9977	0.9965	0.9983	864

混淆矩阵显示，GloVe 将 6 条商业新闻预测为政治、7 条政治新闻预测为商业，两个方向共 13 条，占其 17 条错误中的多数。AG Word2Vec 的对应数量为 9 条和 13 条，共 22 条；NYT Word2Vec 的对应数量为 10 条和 8 条，共 18 条。三组实验的主要混淆均发生在商业与政治之间，与 Task 1 的错误分布特征一致。

(六) 结果分析

GloVe 在三组平均词向量实验中取得最高的 Accuracy 和 Macro-F1，同时具有最高的测试词覆盖率。外部预训练向量所包含的词项及其语义表示，为 NYT 文档提供了较充分的可用特征；但本实验没有单独控制预训练语料规模、词表及向量质量，不能将 GloVe 的优势全部归因于某一因素。

NYT Word2Vec 在三个类别上的 F1 均高于 AG Word2Vec，其词表更大、测试覆盖率更高，且训练语料直接来自目标分类数据的训练部分。结果与目标领域语料能够提供更适合 NYT 的词项表示这一解释相符。不过，两组训练语料的词元总量及文本分布也不同，因此本实验无法将提升单独归因于领域匹配。AG News 的文档数量更多，也未在本次实验中转化为更高的 NYT 分类性能。

与第 三 节相比，三组平均词向量方法的测试 Accuracy 和 Macro-F1 均低于两种词袋方法。平均向量将文档压缩为 100 维表示，能够组合多个单词的语义信息，同时也弱化了独立词项的显式表示，并忽略词序和上下文差异。该表示方式可能影响对细粒度类别线索的保留，但本次实验尚未通过额外对照验证这一机制。当前结果说明，在本数据集及固定配置下，平均静态词向量没有取得优于词袋基线的分类表现。

五、 Task 3：基于 BERT 的文本分类

Task 3 使用 `google-bert/bert-base-uncased` 对 NYT 新闻分类任务进行微调。与前两节的词袋和平均静态词向量不同，BERT 在编码过程中结合上下文生成文本表示，并通过分类损失共同更新编码器与分类头。本实验按照作业要求，将最大序列长度固定为 64，使用完整 NYT 训练集训练 3 轮，并在同一份测试集上评价第三轮最终模型的性能。

(一) 模型结构与分类目标

BERT-base 由 12 层双向 Transformer 编码器组成，隐藏向量维度为 768，每层包含 12 个注意力头。输入表示包含词元、位置和句段信息，自注意力机制使不同位置的词元能够结合其他有效位置的信息更新自身表示。与每个单词具有固定向量的 GloVe 和 Word2Vec 相比，BERT 的词元表示会随输入上下文发生变化。

实验使用 Transformers 的 `AutoModelForSequenceClassification` 加载预训练编码器，并初始化三分类分类头。模型取首位 `[CLS]` 的最终隐藏状态，通过 BERT 的 pooler 进行线性变换和 tanh 激活，再经过 dropout 与分类线性层得到三个类别的 logits：

$$\mathbf{h}_p = \tanh(\mathbf{W}_p \mathbf{h}_{[\mathrm{CLS}]} + \mathbf{b}_p), \tag{14}$$

$$\mathbf{z} = \mathbf{W}_c \,\mathrm{Dropout}(\mathbf{h}_p) + \mathbf{b}_c. \tag{15}$$

其中，$\mathbf{h}_{[\mathrm{CLS}]} \in \mathbb{R}^{768}$，$\mathbf{z} \in \mathbb{R}^3$。分类头的 dropout 概率为 0.1，评价阶段关闭 dropout。类别编号依次为商业 0、政治 1、体育 2，模型配置同时保存类别名称与编号的双向映射。

对一个包含 B 条样本的训练批次，将 logits 经 softmax 转换为类别概率 $p_i(k)$，使用交叉熵损失

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\log p_i(y_i). \tag{16}$$

微调时不冻结编码器，预训练模型参数与分类头参数均参与反向传播。正式实验记录的可训练参数总数为 109,484,547，预测时选择 logits 最大的类别。

(二) 分词、截断与输入构建

实验使用该预训练模型自带的 uncased WordPiece tokenizer。它按照模型规则处理大小写并将文本切分为词元或子词，不使用 Task 1、Task 2 的词袋分词器。对每条新闻，tokenizer 加入 `[CLS]` 和 `[SEP]`，将超长序列截断，将较短序列填充至 64 个位置，同时生成 `attention_mask`，供模型区分有效词元与填充位置。

最大长度 64 包含特殊词元，因此单条新闻最多保留 62 个正文子词位置。这里的子词数量不等于自然语言单词数量，因为一个单词可能被切分成多个 WordPiece。统一截断保证各批次输入形状一致，也意味着较长新闻的后续内容不会进入模型。

代码 3 展示输入编码与分类模型初始化的核心流程。训练、验证和测试集均使用相同的 tokenizer 与最大长度设置，文本与标签依据共享划分中的索引对应。

```
tokenizer = AutoTokenizer.from_pretrained(
    "google-bert/bert-base-uncased", use_fast=True
)
encodings = tokenizer(
    texts[indices].tolist(),
    padding="max_length",
    truncation=True,
    max_length=64,
    return_tensors="pt",
)
model = AutoModelForSequenceClassification.from_pretrained(
    "google-bert/bert-base-uncased",
    num_labels=3,
    label2id=label2id,
    id2label=id2label,
)
```

Listing 3: BERT 输入编码与三分类模型初始化

(三) 微调设置与训练流程

表 9 列出正式微调的主要参数。模型在 NVIDIA Tesla T4 GPU 上运行，使用 CUDA 自动混合精度与 `GradScaler`。梯度反向传播后先进行反缩放，再按最大范数 1.0 裁剪；若混合精度溢出导致参数更新被跳过，则该步不推进学习率调度器。

训练 DataLoader 在每轮打乱训练样本顺序，验证集和测试集保持各自索引顺序，均不参与参数更新。训练集共有 9,215 条新闻，每轮包含 $\lceil 9,215/16 \rceil = 576$ 个批次，三轮计划共 1,728 个优化步。预热步数按比例向下取整为 172，之后学习率随优化步线性下降。

表 9: BERT 正式微调的主要参数

参数	设置
预训练模型	`google-bert/bert-base-uncased`
最大序列长度	64，包含特殊词元
训练轮数	3
训练 / 评价 batch size	16 / 32
优化器	AdamW
基础学习率	2×10^{-5}
权重衰减	0.01；bias 与 LayerNorm 权重不施加衰减
学习率调度	前 10% 的计划优化步预热，随后线性衰减
梯度裁剪最大范数	1.0
随机种子	42
数值计算线程数	4
计算设备与精度	Tesla T4，FP16 混合精度
编码器是否冻结	否，全参数微调

每轮训练结束后，将模型切换到评价模式，在不计算梯度的条件下评价验证集，记录训练损失、验证损失、Accuracy 与 Macro-F1。三个训练轮次均完整执行，不进行提前停止，也不依据验证指标更换最终测试模型。完成第三轮后，使用该轮最终模型进行一次测试，再保存微调权重、tokenizer、类别映射、训练历史及逐条测试预测。

本节采用的记录均来自标记为 `full_experiment` 的完整实验，训练、验证和测试样本数分别为 9,215、1,152 和 1,152，不使用小样本运行检查的指标作为正式结果。

（四） 训练过程与验证表现

表 10 给出三轮训练记录。训练损失是该轮训练期间各批次损失按样本数量加权的平均值，验证损失由该轮结束后的模型在验证集上计算，因此两者对应的模型状态和计算阶段不同。

表 10: BERT 三轮训练的损失与验证指标

轮次	训练损失	验证损失	验证集 Accuracy	验证集 Macro-F1
1	0.2396	0.1547	0.9523	0.8889
2	0.0533	0.0926	0.9800	0.9561
3	0.0167	0.1028	0.9774	0.9533

从第一轮到第二轮，训练损失与验证损失均明显下降，验证 Accuracy 从 0.9523 提高到 0.9800，Macro-F1 从 0.8889 提高到 0.9561，表明模型在微调过程中获得了更好的验证分类表现。第三轮训练损失继续下降，但验证损失有所上升，两个验证指标均小幅回落。

（五） 最终测试结果与类别分析

表 11 给出第三轮最终模型的验证集与测试集结果。

表 11: BERT 第三轮最终模型的分类结果

数据子集	Accuracy	Macro-F1
验证集	0.9774	0.9533
测试集	0.9809	0.9577

BERT 在 1,152 条测试新闻中正确分类 1,130 条，错误分类 22 条。表 12 列出每类指标，按照商业、政治、体育的行列顺序，其混淆矩阵为

$$\mathbf{M}_{\text{BERT}} = \begin{bmatrix} 136 & 5 & 2 \\ 8 & 134 & 3 \\ 2 & 2 & 860 \end{bmatrix}. \tag{17}$$

表 12: BERT 的测试集每类分类指标

类别	Precision	Recall	F1	样本数
商业	0.9315	0.9510	0.9412	143
政治	0.9504	0.9241	0.9371	145
体育	0.9942	0.9954	0.9948	864

体育类正确识别 860 条, F1 为 0.9948, 仍然高于商业和政治两类。政治类的召回率为 0.9241, 145 条样本中有 11 条被误分类，其中 8 条预测为商业、3 条预测为体育。商业与政治两个方向的混淆共 13 条，占全部 22 条错误中的多数，说明这两类仍是模型的主要分类难点。

由于体育类占测试集的大多数，且其分类表现较好，整体 Accuracy 高于对三个类别等权平均的 Macro-F1。BERT 的上下文表示并未在本次设置下消除类别之间的表现差异，因此仍需结合每类指标评价其效果。

(六) 与传统方法的比较及局限

BERT 的测试 Accuracy 为 0.9809，Macro-F1 为 0.9577，两个指标均高于 AG Word2Vec，但低于 GloVe 和两种词袋方法。与 NYT Word2Vec 相比，BERT 的 Accuracy 略低，Macro-F1 略高，表明两种指标对类别表现的侧重不同，不能仅凭其中一个指标给出完全相同的排序。

输入长度是本次比较中的一个重要条件。Task 1 与 Task 2 处理完整新闻的词元，而 BERT 固定只接收最多 64 个包含特殊词元的位置。长新闻截断后，部分内容无法参与分类，这可能影响其可利用的类别信息。不过，本实验没有测量截断比例，也没有进行不同长度的对照，因而不能将 BERT 未超过词袋方法的结果直接归因于截断。

此外，BERT 采用固定学习率、batch size 和 3 轮训练配置，未针对验证集进一步搜索超参数。第三轮验证表现的小幅回落也提示，训练轮次与泛化表现之间需要结合具体数据观察。这些因素为后续改进提供了方向，但本次结果仅说明该模型在当前数据、划分及训练条件下的表现，不足以判断预训练语言模型在其他配置或数据集上的性能。